

CI/CD Security & Software Supply Chain

Why pipeline security matters

If an attacker can modify your build, they own every customer who installs the product. Supply chain attacks (SolarWinds 2020, XZ utils 2024) show the blast radius: poison once, consume forever.

Attackers' favorite moves

Dependency confusion – publish a malicious package with the same name as an internal one, with a higher version, to public registries

Typosquatting – a similarly-named package ("requests-ssl")

Stolen CI tokens – exfiltrated env vars in build logs

Compromised upstream – a legit dependency gets backdoored

Build-time code injection – PRs with malicious workflow changes

Core defenses

Pin exact dependency versions; lockfiles committed; verify checksums

Secrets never in the repo: use the secret manager + inject at runtime; rotate leaked tokens

Least-privilege CI tokens; short-lived credentials

Signed artifacts – sign builds; consumers verify (SLSA levels)

Require PR approval + signed commits for release branches

Scope dependency feeds: allow trusted registries/namespaces only

Monitor advisories → **SBOM** matching to your inventory

SLSA in one paragraph

Supply-chain Levels for Software Artifacts (salsa.dev): four levels describing provenance integrity – from "nothing" (L0) to "hosted auditable builds with signed provenance" (L3/L4). Aim L3+ for release artifacts.

IaC as part of pipeline

Scan Terraform/CloudFormation/K8s manifests in PR (checkov, tfsec, KICS)

Gate on misconfigurations (world-open S3, admin-users, egress everything)

Enforce drift detection: prod must match reviewed IaC.

Cold realism: the hardest part is not the tooling – it is writing a pipeline where a compromised dependency or a tired engineer cannot silently ship malicious code.

[← SAST, DAST & SCA: The AppSec...](#)

[AppSec Review & Threat Modelin...](#)